

Требования и документация

Выявление, классификация, критерии качества, приоритизация и трассируемость требований. Документные форматы: SRS, ГОСТ 34, user story, use case.

Редакция от 06.08.2026

Содержание

1. Что такое требование
 - 1.1 Уровни требований
 - 1.2 Признак хорошо поставленной цели
2. Виды требований подробно
 - 2.1 Функциональные требования
 - 2.2 Нефункциональные требования
 - 2.3 Ограничения
3. Выявление требований
 - 3.1 Методы и когда они работают
 - 3.2 Техника интервью
 - 3.3 Работа со стейкхолдерами
4. Критерии качества требований
 - 4.1 Слова-маркеры плохого требования
5. Приоритизация
6. Документные форматы
 - 6.1 SRS
 - 6.2 Техническое задание по ГОСТ 34.602
 - 6.3 User story и критерии приёмки
 - 6.4 Use case по Коберну
 - 6.5 Выбор формата
7. Трассируемость и изменения
 - 7.1 Матрица трассируемости
 - 7.2 Управление изменениями
8. Проверка требований
 - 8.1 Верификация и валидация
 - 8.2 Ревью требований
 - 8.3 Чек-лист дефектов требований
9. Глоссарий и матрица прав
 - 9.1 Глоссарий
 - 9.2 Матрица ролей и прав

1. Что такое требование

Требование это утверждение о том, что система должна делать или каким свойством обладать, записанное так, что его выполнение можно проверить. Последняя часть определения важнее первой: утверждение, которое нельзя проверить, требованием не является, каким бы разумным оно ни выглядело.

1.1 Уровни требований

Требования живут на разных уровнях абстракции, и смешивать их в одном документе означает гарантированную путаницу.

Уровень	Отвечает на вопрос	Кто автор	Пример
Бизнес-требование	Зачем компании этот проект	Заказчик, спонсор	Сократить срок оформления пропуска с двух дней до четырёх часов
Требование стейкхолдера	Что нужно конкретной группе пользователей	Пользователи	Охранник должен видеть, кому разрешён проход прямо сейчас
Функциональное требование	Что делает система	Аналитик	Система фиксирует отметку въезда с указанием поста и времени
Нефункциональное требование	Каким свойством обладает	Аналитик, архитектор	Список из десяти тысяч записей открывается не дольше двух секунд
Ограничение	Чего нельзя менять	Заказчик, архитектура	Хранение данных только на серверах в России
Бизнес-правило	Какое правило предметной области действует	Заказчик	Заявку с истёкшим сроком вложения дополнить нельзя

Бизнес-правило существует независимо от системы: оно действовало бы и при бумажном документообороте. Функциональное требование описывает, как система это правило соблюдает. Различать полезно потому, что бизнес-правила меняются реже кода и живут в отдельном перечне.

1.2 Признак хорошо поставленной цели

Бизнес-требование должно содержать измеримый результат. Формулировка «повысить эффективность работы бюро» не даёт понять, достигнут результат или нет. Формулировка «сократить среднее время от подачи заявки до выдачи пропуска с 14 часов до 4» даёт

критерий, по которому через полгода можно честно ответить, окупился проект или нет.

2. Виды требований подробно

2.1 Функциональные требования

Описывают поведение: что система принимает на вход, что делает, что отдаёт, как реагирует на ошибки. Форматов записи три, и выбор зависит от процесса в команде.

Классическая форма «система должна». Подходит для формальных документов и госзаказа.

Система должна отклонять отправку заявки, если хотя бы один её объект точно совпадает с действующей записью чёрного списка, и показывать заявителю причину отказа с указанием совпавшего объекта.

User story. Подходит для гибких процессов.

Как заявитель, я хочу видеть причину отказа при отправке заявки, чтобы исправить данные и не обращаться в бюро.

Use case. Подходит для сложных сценариев с ветвлениями.

Ошибка новичка: считать эти формы взаимозаменяемыми. User story это обещание разговора и напоминание о ценности, она не заменяет описание поведения. Полное требование это история плюс критерии приёмки плюс, при необходимости, описание сценария.

2.2 Нефункциональные требования

Самая недооценённая часть работы. Систему принимают по функциям, а отказываются от неё из-за нефункциональных свойств.

Две рабочие классификации. **FURPS+**: функциональность, удобство использования, надёжность, производительность, поддерживаемость, плюс ограничения (проектные, реализационные, интерфейсные, физические). **ISO/IEC 25010**: восемь характеристик качества, среди которых функциональная пригодность, производительность, совместимость, удобство, надёжность, защищённость, сопровождаемость, переносимость.

Ключевой навык: превращать пожелание в проверяемое число.

Пожелание заказчика	Требование
Должно работать быстро	95-й перцентиль времени ответа списка заявок не превышает 500 мс при 300 одновременных пользователях
Должно быть надёжно	Плановая доступность 99,5% в рабочие часы, восстановление после сбоя не дольше 30 минут

Пожелание заказчика	Требование
Должно выдерживать нагрузку	1500 одновременных сессий, 40 запросов в секунду в пике, рост данных 2 млн записей в год
Должно быть безопасно	Доступ по ролям, журнал обращений к персональным данным с хранением 3 года, пароли по политике не короче 12 символов
Должно быть удобно	Новый охранник выполняет отметку прохода без обучения не дольше чем за 15 секунд

Вопросы, которые задают заказчику, чтобы вытащить нефункциональные требования:

- Сколько человек будет работать одновременно в пиковый час и в обычный?
- Сколько записей накопится за год, за три года?
- Что произойдёт с работой предприятия, если система будет недоступна час? Сутки?
- Есть ли часы, когда простой недопустим совсем?
- Какие данные считаются персональными и кто имеет право их видеть?
- Из каких сетей и с каких устройств будут работать?
- Есть ли требования регуляторов к хранению и к месту размещения данных?

2.3 Ограничения

Ограничение отличается от требования тем, что не обсуждается. Технологический стек, площадка размещения, сроки, бюджет, обязательная интеграция с существующей системой, требования законодательства. Фиксируются отдельным разделом, чтобы к ним не возвращались на каждой встрече.

3. Выявление требований

3.1 Методы и когда они работают

Метод	Когда применять	Ограничение
Интервью	Почти всегда, основной метод	Человек описывает работу идеализированно
Наблюдение за работой	Процесс сложный или неформализованный	Дорого по времени, эффект наблюдателя
Анализ документов	Есть регламенты, инструкции, формы	Документы часто расходятся с практикой
Анализ существующей системы	Замена или доработка	Легко унаследовать чужие ошибки
Воркшоп	Нужно согласовать позиции нескольких сторон	Требуется фасилитация, иначе спор
Прототип	Заказчик не может описать желаемое словами	Прототип принимают за готовое решение
Опрос	Много пользователей, нужны цифры	Не даёт понимания причин
Анализ данных системы	Нужно понять реальное поведение	Показывает что, но не почему

3.2 Техника интервью

Правила, которые отличают полезное интервью от разговора ни о чём:

Готовиться заранее. Прочитать доступные документы и посмотреть систему до встречи. Вопрос «а что вы вообще делаете» тратит время собеседника и обесценивает встречу.

Начинать с открытых вопросов. «Расскажите, как вы сейчас оформляете заявку» даёт больше, чем «правильно ли, что вы оформляете заявку через форму». Закрытые вопросы оставить на уточнение деталей.

Спрашивать про исключения отдельно. Основной сценарий человек расскажет сам. Про отказы, ошибки и редкие случаи придётся спросить прямо: «а если человек приехал без документов?», «а если заявку подали, а машина сломалась?», «что делаете, когда система недоступна?».

Идти от потребности, а не от решения. Услышав «нам нужна кнопка выгрузки», спросить, что человек делает с выгруженным файлом. Часто выясняется, что файл открывают, считают сумму и отправляют письмом, и настоящая потребность в другом.

Техника пяти почему. Последовательно спрашивать «почему» до тех пор, пока не дойдёте до причины, а не до симптома. Обычно хватает трёх шагов, пять это метафора.

Фиксировать формулировки собеседника. Термины предметной области записывать дословно: они пойдут в глоссарий. Переименование сущности аналитиком по своему вкусу рождает недопонимание на годы вперёд.

Заканчивать пересказом. «Давайте я повторю, как понял» ловит половину расхождений на месте, пока их дёшево исправить.

3.3 Работа со стейкхолдерами

Стейкхолдер это любой, кто влияет на систему или испытывает её влияние. Их перечень составляют явно, иначе на приёмке появится подразделение, о котором не подумали.

Матрица влияния и интереса даёт стратегию работы с каждым: с высоким влиянием и высоким интересом работать плотно, с высоким влиянием и низким интересом держать в курсе, с низким влиянием и высоким интересом информировать, остальных наблюдать.

Отдельно стоит держать в голове, что пользователь и заказчик это разные люди с разными интересами. Заказчик платит и хочет контроля, пользователь работает и хочет, чтобы ему не мешали. Требования, собранные только у руководителя, дают систему, в которой неудобно работать.

4. Критерии качества требований

Требование проверяют по семи признакам. Проверка занимает минуту и экономит недели.

Полнота. Описано поведение во всех значимых случаях, включая отказы. Нет висящих ссылок на неопределённые сущности.

Непротиворечивость. Не конфликтует с другими требованиями. Конфликты бывают неявными: одно требование обещает мгновенный ответ, другое требует синхронной проверки по внешнему реестру.

Однозначность. Два человека прочтут одинаково. Враги однозначности: «и/или», «и т.п.», «при необходимости», «должен корректно обрабатывать», «дружественный интерфейс».

Проверяемость. Существует способ убедиться, что требование выполнено. Если нельзя придумать тест, требование нужно переписать.

Реализуемость. Выполнимо в рамках технологии, бюджета и сроков.

Необходимость. За требованием стоит потребность, а не привычка. Вопрос «что произойдёт, если этого не будет» отсеивает многое.

Трассируемость. Видно, откуда требование взялось и во что превратилось.

4.1 Слова-маркеры плохого требования

Формулировки, при виде которых требование стоит переписать: «удобно», «быстро», «гибко», «эффективно», «интуитивно понятно», «по возможности», «стандартным образом», «как обычно», «аналогично существующему», «и другие», «при необходимости», «система должна корректно работать».

Отдельная ловушка это слово «должна» в цепочке: «система должна проверить и, если данные корректны, сохранить и отправить уведомление, иначе показать ошибку». Такое предложение содержит четыре требования, и тестировать его как одно невозможно. Разбивать.

5. Приоритизация

Требований всегда больше, чем времени. Отказ от приоритизации означает, что порядок выберет тот, кто громче просит.

MoSCoW. Must have (без этого запуск бессмысленен), Should have (важно, но можно отложить), Could have (сделаем, если останется время), Won't have (не в этот раз, фиксируем явно). Простой и понятный заказчику метод. Главная опасность: заказчик помечает всё как Must. Лечится ограничением: Must не больше 60% объёма.

Kano. Делит функции на базовые (их отсутствие раздражает, наличие не радует), линейные (чем больше, тем лучше) и восхищающие (о них не просили, но они создают впечатление). Полезен для продуктовых решений.

RICE. Reach (охват), Impact (влияние), Confidence (уверенность), Effort (трудозатраты). Числовая оценка, удобна при сравнении несопоставимых задач.

WSJF. Стоимость задержки, делённая на длительность. Применяется в масштабируемых гибких методологиях.

Практический совет: метод менее важен, чем сам факт разговора о приоритетах с заказчиком. Задача приоритизации не в точном числе, а в том, чтобы вслух проговорить, от чего готовы отказаться.

6. Документные форматы

6.1 SRS

Спецификация требований к программному обеспечению, классическая структура по ISO/IEC/IEEE 29148:

1. Введение: назначение, границы, термины, ссылки.
2. Общее описание: контекст системы, функции верхнего уровня, пользователи, ограничения, допущения.
3. Требования: функциональные по подсистемам, внешние интерфейсы, нефункциональные.
4. Приложения: глоссарий, модели, матрица трассируемости.

Полный SRS уместен, когда система заказывается у подрядчика по договору. В продуктовой команде документ такого размера устаревает быстрее, чем дочитывается.

6.2 Техническое задание по ГОСТ 34.602

Обязательный формат для государственных заказчиков, банков с госучастием, оборонных предприятий и крупных интеграторов. Структура жёсткая:

1. Общие сведения.
2. Назначение и цели создания системы.
3. Характеристика объекта автоматизации.
4. Требования к системе: к системе в целом, к функциям, к видам обеспечения.
5. Состав и содержание работ по созданию системы.
6. Порядок контроля и приёмки системы.
7. Требования к составу и содержанию работ по подготовке объекта к вводу в действие.
8. Требования к документированию.
9. Источники разработки.

Полезно знать даже тем, кто идёт в продукт: структура ГОСТ это хороший чек-лист того, о чём стоит подумать. Раздел «порядок контроля и приёмки» задаёт вопрос, который в гибких командах часто забывают: как мы поймём, что работа принята.

6.3 User story и критерии приёмки

Формат истории: как <роль>, я хочу <действие>, чтобы <ценность>. Третья часть обязательна и чаще всего пропускается. Без неё непонятно, зачем функция, и невозможно предложить более дешёвую альтернативу.

Качество истории проверяют по **INVEST**: независима, обсуждаема, ценна, оцениваема, мала, тестируема.

Критерии приёмки записывают в формате Gherkin:

Сценарий: отправка заявки с объектом из чёрного списка

Дано заявка содержит человека, точно совпадающего с действующей записью чёрного списка

Когда заявитель нажимает "Отправить"

Тогда заявка не создаётся

И заявитель видит причину отказа с указанием совпавшего человека

И в журнале появляется запись о попытке с указанием заявителя и времени

Правило: критерии пишутся до начала разработки и согласуются с тестировщиком. Критерии, дописанные после демонстрации, это не критерии, а объяснение того, что получилось.

6.4 Use case по Коберну

Формат для сложных сценариев. Шаблон:

Поле	Содержание
Название	Глагол плюс объект: «Принять заявку в работу»
Основной актер	Кто инициирует
Заинтересованные стороны	Кто ещё имеет интерес в исходе
Предусловия	Что верно до начала
Гарантии успеха	Что верно после успешного завершения
Минимальные гарантии	Что верно, даже если сценарий прерван
Триггер	Событие, запускающее сценарий
Основной сценарий	Пронумерованные шаги без ветвлений
Расширения	Ветвления и исключения с привязкой к номеру шага

Сила формата в разделе расширений. Он вынуждает пройти по каждому шагу основного сценария и спросить, что может пойти не так. Пример: шаг 3 «система проверяет объекты по чёрным спискам» порождает расширения 3a (точное совпадение), 3b (близкое совпадение), 3c (сервис проверки недоступен).

6.5 Выбор формата

Ситуация	Формат
Гибкая продуктовая команда	User story с критериями приёмки
Сложный сценарий с ветвлениями	Use case плюс диаграмма последовательности

Ситуация	Формат
Заказная разработка по договору	SRS или ТЗ по ГОСТ
Интеграция с внешней системой	Отдельная постановка на интеграцию с контрактом
Изменение существующего поведения	Описание «было / стало» с явным указанием, что не меняется

Последняя строка отвечает на частый вопрос новичка: описывать ли поведение целиком или только изменения. Ответ зависит от того, есть ли актуальное описание текущего поведения. Если нет, «стало» без «было» не поймёт ни разработчик, ни тестировщик.

7. Трассируемость и изменения

7.1 Матрица трассируемости

Связывает бизнес-требование, функциональное требование, элемент реализации и тест. Отвечает на два вопроса: что сломается, если изменить это требование, и покрыто ли требование тестом.

В полном виде матрицу ведут в регулируемых отраслях. В обычной команде роль матрицы выполняют ссылки: задача ссылается на требование, коммит на задачу, тест на критерий приёмки. Этого достаточно, если ссылки проставляют дисциплинированно.

Проверка, что трассируемость работает: возьмите произвольную строку кода и попробуйте за три шага дойти до причины, по которой она написана. Если не получается, история решений потеряна.

7.2 Управление изменениями

Требования меняются, и это нормально. Ненормально, когда изменение проходит незамеченным. Минимальный порядок:

1. Изменение фиксируется письменно, с автором и датой.
2. Оценивается влияние: какие требования, какие уже сделанные части, какие сроки.
3. Согласуется с тем, кто отвечает за сроки и бюджет.
4. Обновляется документ, а не только чат.

Отдельно стоит держать журнал решений: что рассматривали, что выбрали, почему отказались от альтернативы. Это самый дешёвый способ не переигрывать одно и то же решение дважды в год.

8. Проверка требований

8.1 Верификация и валидация

Верификация отвечает на вопрос, правильно ли записано требование: непротиворечиво, однозначно, проверяемо. Это внутренняя проверка качества документа.

Валидация отвечает на вопрос, то ли это требование: решает ли оно настоящую задачу заказчика. Проверяется только с заказчиком и пользователями.

Документ может пройти верификацию идеально и провалить валидацию полностью. Безупречно описанная функция, которая никому не нужна, это провал аналитика, а не успех.

8.2 Ревью требований

Формы ревью по возрастанию строгости: вычитка коллегой, обсуждение в команде, формальная инспекция с ролями и протоколом.

Практичный приём: попросить тестировщика прочитать постановку до начала разработки. Тестировщик профессионально ищет дыры и найдёт их за полчаса вместо двух недель.

8.3 Чек-лист дефектов требований

Прогонять по своему тексту перед отправкой:

- Есть требование без указания, кто выполняет действие.
- Есть слово из списка маркеров неоднозначности.
- Описан только успешный сценарий.
- Не указано поведение при недоступности внешней системы.
- Не указано, что происходит с уже существующими данными при изменении правил.
- Не указаны права: кто имеет доступ к функции и к данным.
- Не указано, что попадает в журнал и историю.
- Есть числа без единиц измерения или без указания, откуда взяты.
- Есть ссылка на «существующую логику» без ссылки на её описание.
- Не описано поведение при одновременной работе двух пользователей с одним объектом.
- Не указан срок хранения данных и порядок удаления.
- Термин употреблён в двух значениях в одном документе.

9. Глоссарий и матрица прав

9.1 Глоссарий

Самый недооценённый артефакт. Половина конфликтов в требованиях это спор о значении слова, который никто не заметил.

Правила ведения: одно значение на термин, термины предметной области берутся из речи заказчика, синонимы указываются явно и сводятся к одному основному, в документах используется только основной вариант.

Показательный пример из практики: в системе пропусков слово «согласующий» и слово «принимающий» обозначают роли с разными полномочиями, при этом в коде одна из них называется словом approver. Такое расхождение стоит зафиксировать в глоссарии явно, иначе новый разработчик реализует не ту роль.

9.2 Матрица ролей и прав

Таблица «роль на строке, действие в столбце». Заполняется до разработки, потому что права почти всегда влияют на модель данных.

Действие	Заявитель	Бюро	Согласующий	Охранник	Администратор
Подать заявку	да	да	да	нет	да
Принять в работу	нет	да	нет	нет	да
Проголосовать	нет	нет	да	нет	нет
Отметить проход	нет	нет	нет	да	да
Видеть чужие заявки	нет	да	только свои на согласовании	нет	да
Управлять правами	нет	нет	нет	нет	да

Вопросы, которые матрица заставляет задать: видит ли роль объект целиком или частично, различаются ли права на чтение и изменение, что происходит с доступом при смене должности, кто имеет право выдать право.